

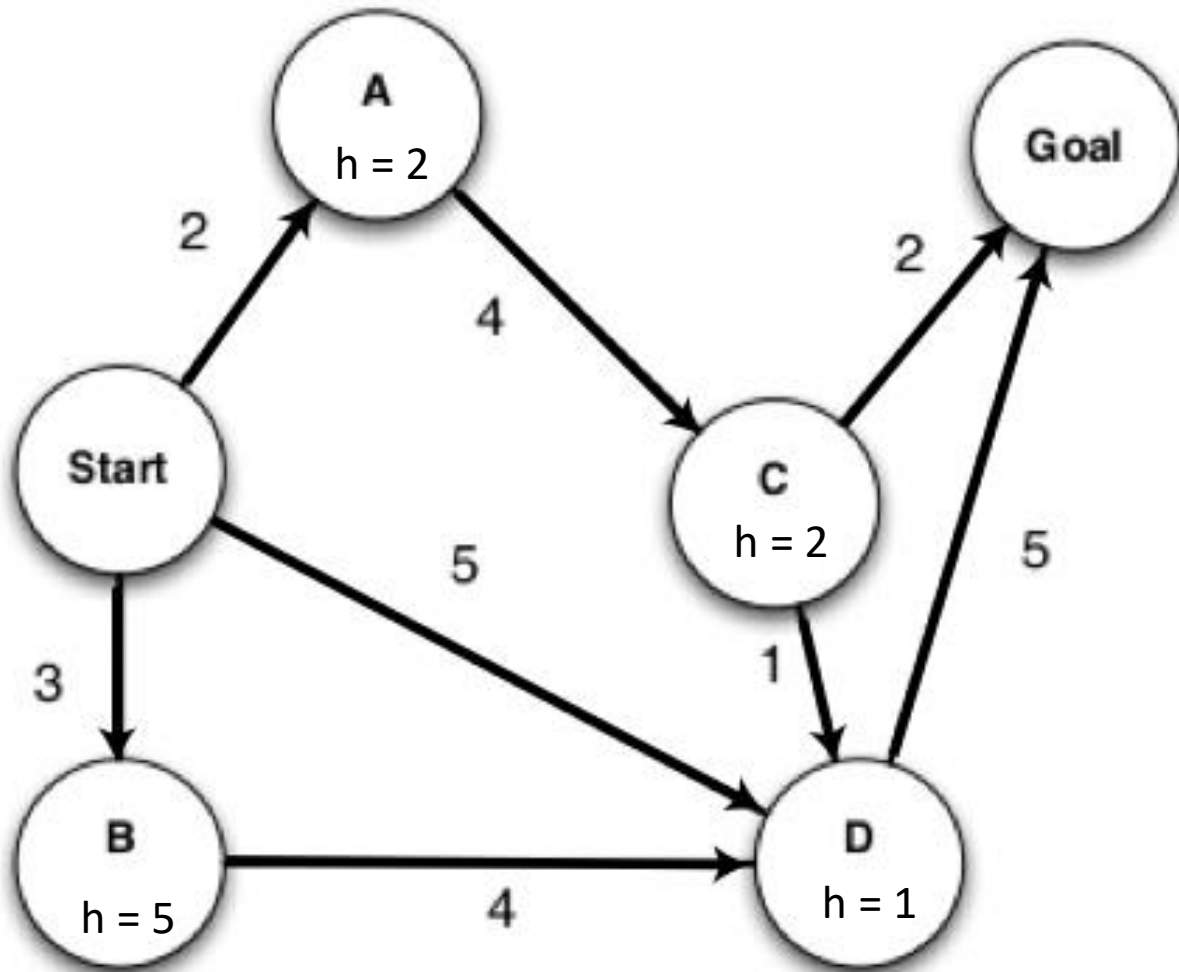
# 3. Adversarial Search

Ack: Berkeley AI course

# Tree Search

- Uninformed Search
  - Depth First Search
  - Breadth First Search
  - Uniform Cost Search (*backward cost*  $g(n)$ )
- Heuristic Search
  - Greedy search (*forward cost*  $h(n)$ )
  - A\* search ( $f(n) = g(n) + h(n)$ , admissible / consistent heuristics)

# Exercise



DFS:

BFS:

UCS:

Greedy:

A\*:

# Outline

- Games
- Adversarial games
- Adversarial search
  - minmax
  - expectimax

# Agents Getting Along with Others



# Games

- Many different kinds of games!
- Types of games:
  - Deterministic or stochastic?
  - One, two, or more players?
  - Perfect information (can you see the state)?
  - Zero sum?

# Zero Sum

- Zero-Sum Games

- Agents have opposite utilities (values on outcomes)
- Let us a single value that one maximizes and the other minimizes
- Adversarial, pure competition

- General Games

- Agents have independent utilities (values on outcomes)
- Cooperation, indifference, competition, and more are all possible

# Zero-Sum Game Games

- **Checkers:** 1950: First computer player. 1994: First computer champion: Chinook ended 40-year-reign of human champion Marion Tinsley using complete 8-piece endgame. 2007: Checkers solved!
- **Chess:** 1997: Deep Blue defeats human champion Gary Kasparov in a six-game match. Deep Blue examined 200M positions per second. Current programs are even better, if less historic.
- **Go: 2016: Alpha GO defeats human champion. Uses Monte Carlo Tree Search, learned evaluation function.**

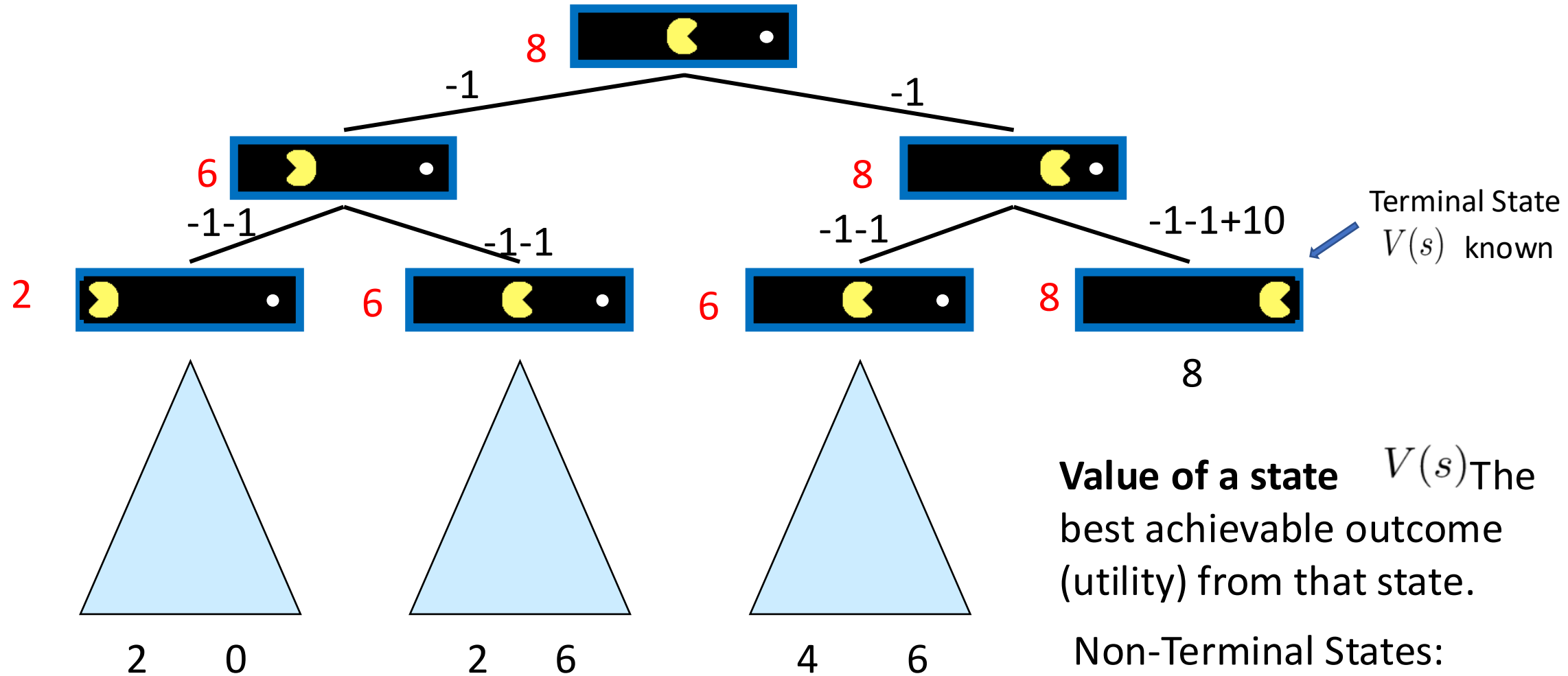
# Deterministic Games with Terminal Utilities

- Many possible formalizations, one is:
  - States:  $S$  (start at  $s_0$ )
  - Players:  $P=\{1\dots N\}$  (usually take turns)
  - Actions:  $A$  (may depend on player / state)
  - Transition Function:  $S \times A \rightarrow S$
  - Terminal Test:  $S \rightarrow \{t, f\}$
  - Terminal Utilities:  $S \times P \rightarrow R$
- Solution for a player is a **policy**:  $S \rightarrow A$

# Outline

- Games
- **Adversarial games**
- Adversarial search
  - minmax
  - expectimax

# Single-Agent Trees

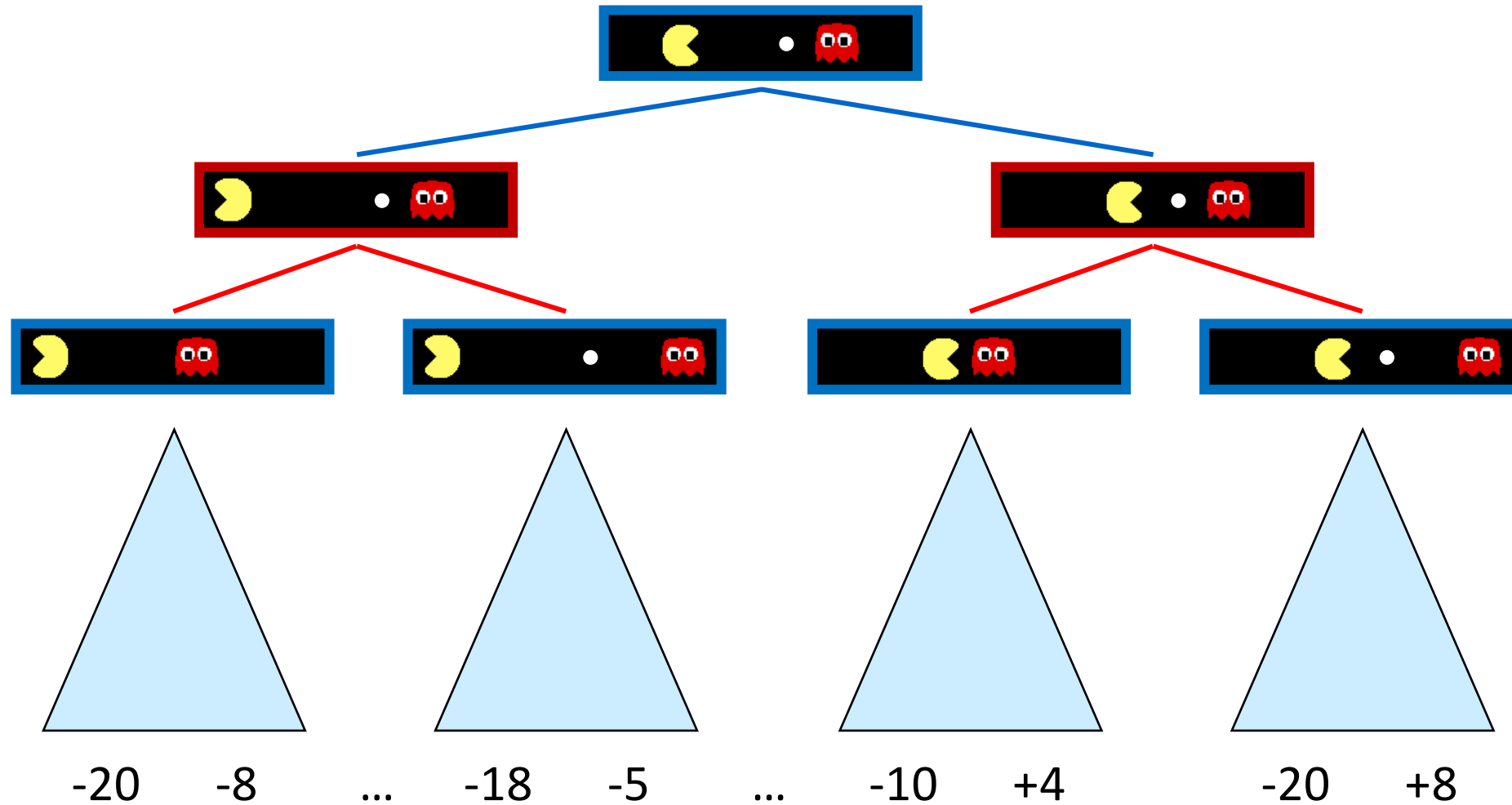


**Value of a state**  $V(s)$  The best achievable outcome (utility) from that state.

### Non-Terminal States:

$$V(s) = \max_{s' \in \text{children}(s)} V(s')$$

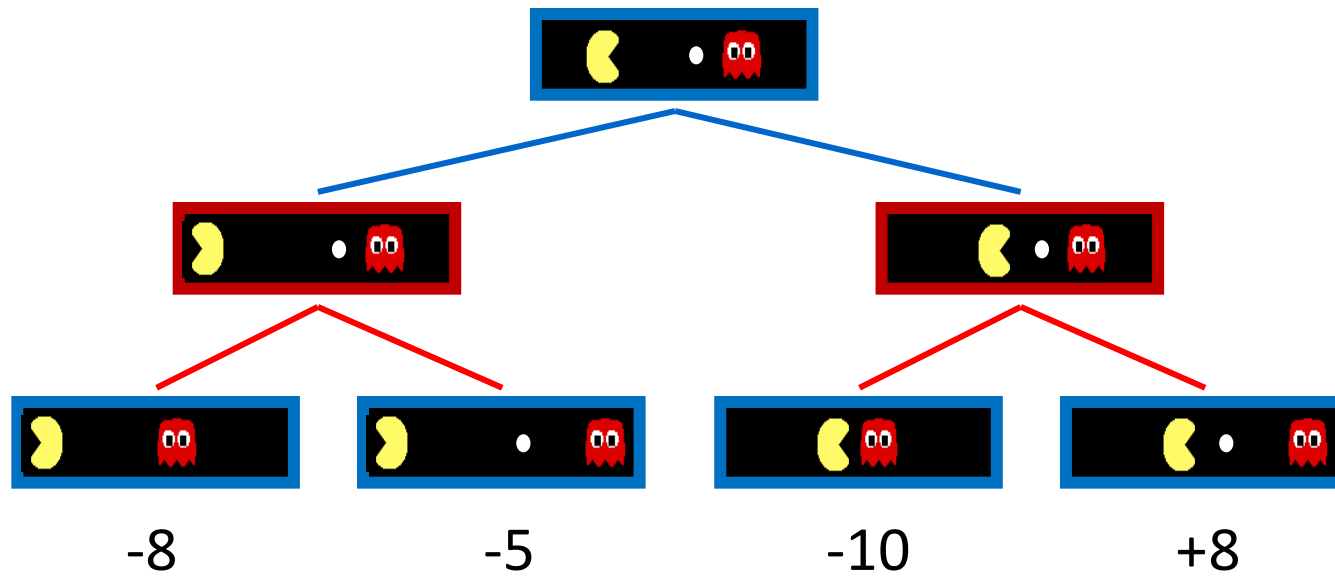
# Adversarial Game Trees



# Outline

- Games
- Adversarial games
- **Adversarial search**
  - minmax
  - expectimax

# Minimax

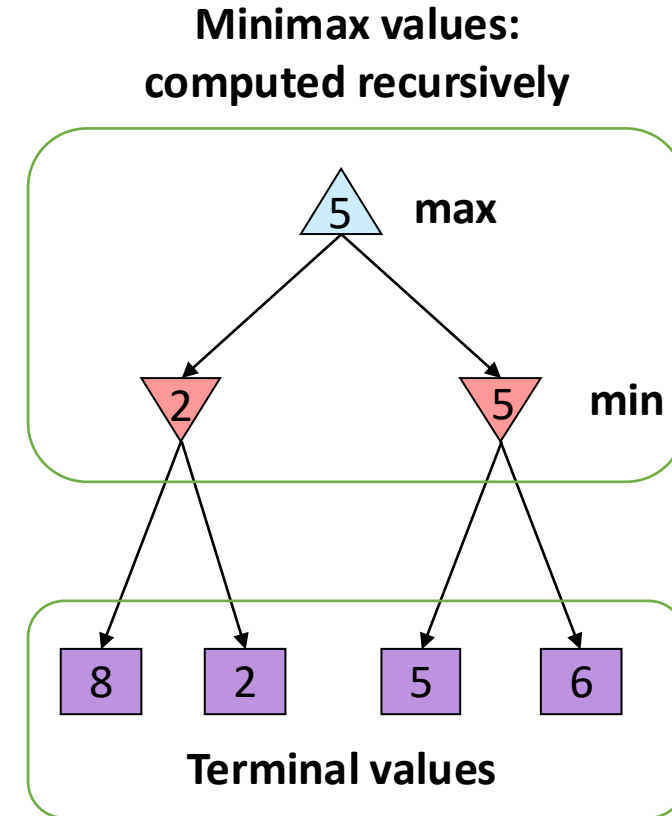


Agent:  $V(s) = \max_{s' \in \text{successors}(s)} V(s')$

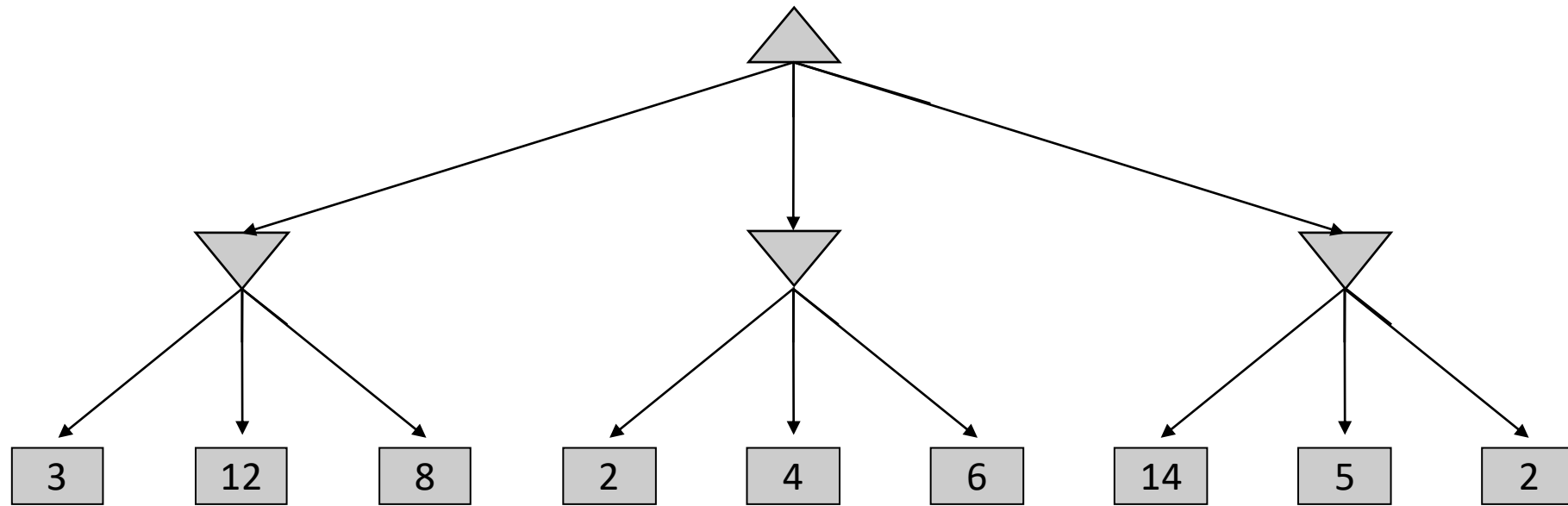
Opponent:  $V(s') = \min_{s \in \text{successors}(s')} V(s)$

# Adversarial Search (Minimax)

- Deterministic, zero-sum games:
  - Tic-tac-toe
  - One player maximizes result
  - The other minimizes result
- Minimax search:
  - A state-space search tree
  - Players alternate turns
  - Compute each node's **minimax value**: the best achievable utility against a rational (optimal) adversary



# Minimax Example



# Minimax Implementation

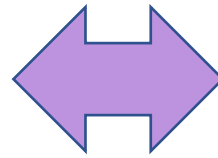
def max-value(state):

    initialize  $v = -\infty$

    for each successor of state:

$v = \max(v, \text{min-value}(\text{successor}))$

    return  $v$



def min-value(state):

    initialize  $v = +\infty$

    for each successor of state:

$v = \min(v, \text{max-value}(\text{successor}))$

    return  $v$

# Minimax Implementation (Dispatch)

```
def value(state):
```

if the state is a terminal state: return the state's utility

if the next agent is MAX: return `max-value(state)`

if the next agent is MIN: return `min-value(state)`

```
def max-value(state):
```

initialize  $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return  $v$

```
def min-value(state):
```

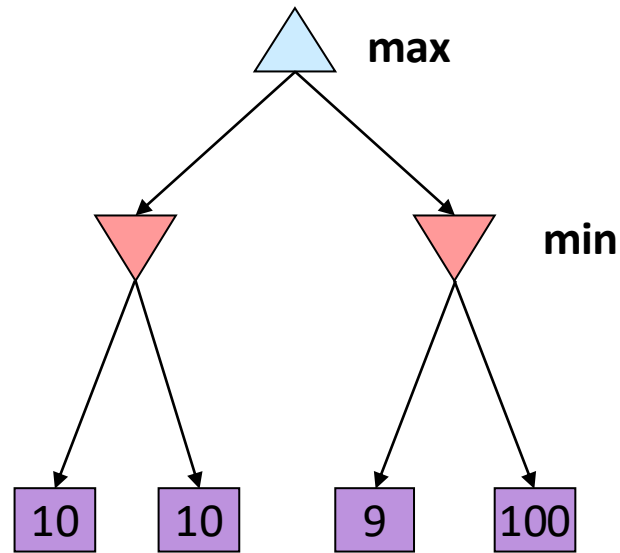
initialize  $v = +\infty$

for each successor of state:

$v = \min(v, \text{value}(\text{successor}))$

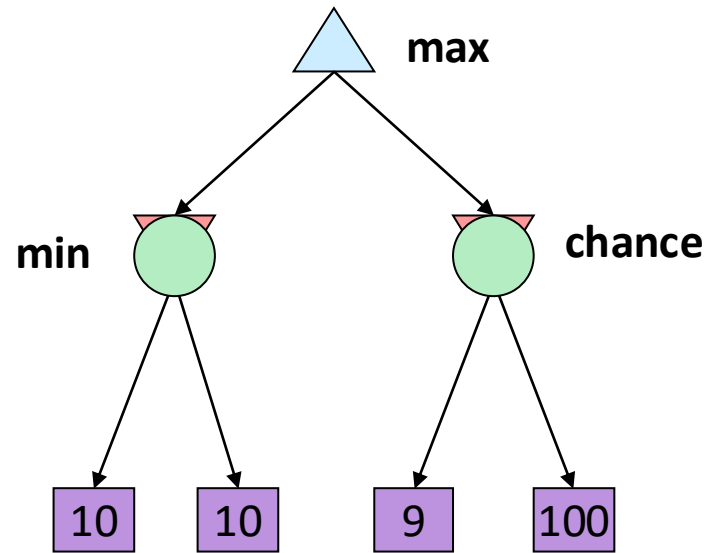
return  $v$

# Minimax Properties



Optimal against a perfect player. Otherwise?

# Worst-Case vs. Average Case



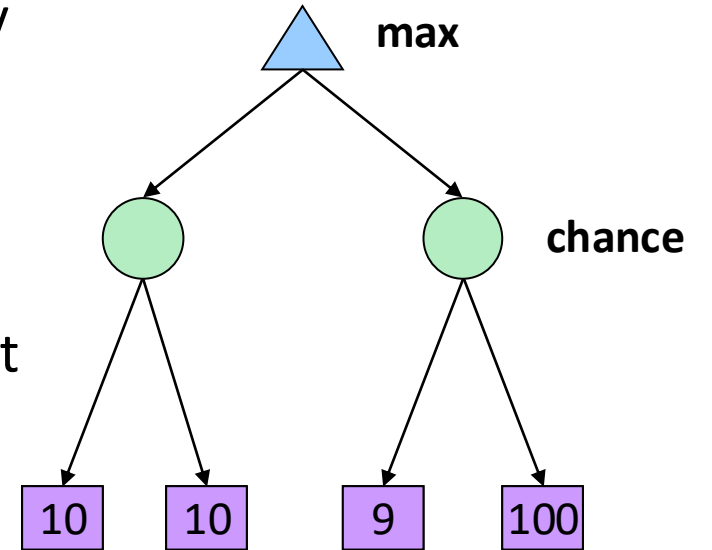
Idea: Uncertain outcomes controlled by chance, not an adversary!

# Why not minimax?

- **Worst case reasoning is too conservative**
- Why wouldn't we know what the result of an action will be?
  - Explicit randomness: rolling dice
  - Unpredictable opponents: the ghosts respond randomly
  - Unpredictable humans: humans are not perfect
  - Actions can fail: when moving a robot, wheels might slip
- Values should now reflect average-case (expectimax) outcomes, not worst-case (minimax) outcomes

# Expectimax Search

- Expectimax search: compute the **average score** under optimal play
  - **Max** nodes as in minimax search
  - **Chance** nodes are like min nodes but the outcome is uncertain
  - Calculate their **expected utilities**
  - I.e. take weighted average (**expectation**) of children
- Later, we'll learn how to formalize the underlying uncertain-result problems as **Markov Decision Processes**



# Expectimax Pseudocode

```
def value(state):
```

if the state is a terminal state: return the state's utility

if the next agent is MAX: return max-value(state)

if the next agent is EXP: return exp-value(state)

```
def max-value(state):
```

initialize  $v = -\infty$

for each successor of state:

$v = \max(v, \text{value}(\text{successor}))$

return v

```
def exp-value(state):
```

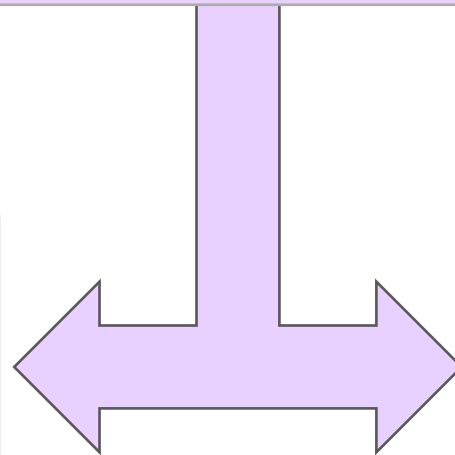
initialize  $v = 0$

for each successor of state:

$p = \text{probability}(\text{successor})$

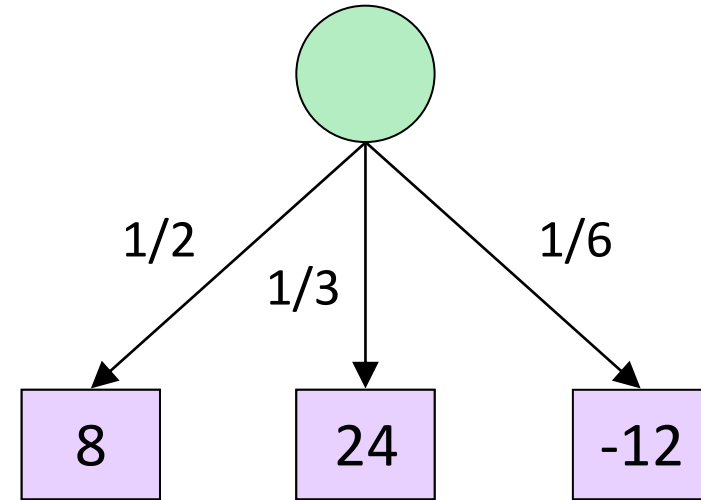
$v += p * \text{value}(\text{successor})$

return v



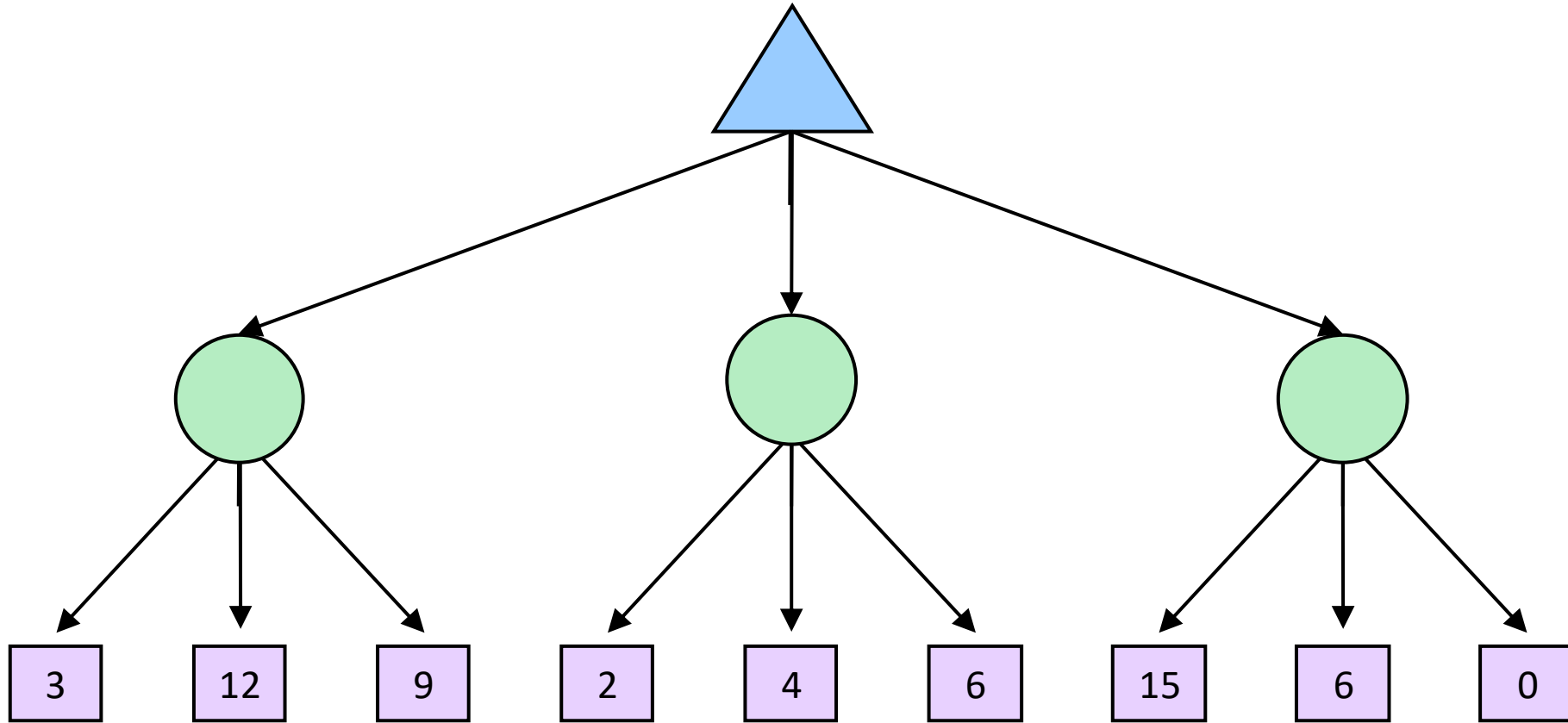
# Expectimax Pseudocode

```
def exp-value(state):  
    initialize v = 0  
    for each successor of state:  
        p = probability(successor)  
        v += p * value(successor)  
    return v
```



$$v = (1/2) (8) + (1/3) (24) + (1/6) (-12) = 10$$

# Expectimax Example



# Probabilities

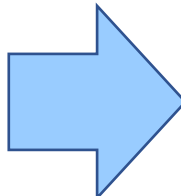
- A **random variable** represents an event whose outcome is unknown
- A **probability distribution** is an assignment of weights to outcomes
- Example: Traffic on freeway
  - Random variable:  $T$  = whether there's traffic
  - Outcomes:  $T$  in {none, light, heavy}
  - Distribution:  $P(T=\text{none}) = 0.25$ ,  $P(T=\text{light}) = 0.50$ ,  $P(T=\text{heavy}) = 0.25$

# Probabilities

- Some laws of probability (more later):
  - Probabilities are always non-negative
  - Probabilities over all possible outcomes sum to one
- As we get more evidence, probabilities may change:
  - $P(T=\text{heavy}) = 0.25$ ,  $P(T=\text{heavy} \mid \text{Hour}=8\text{am}) = 0.60$
  - We'll talk about methods for reasoning and updating probabilities later

# Expectations

- The expected value of a function of a random variable is the average, weighted by the probability distribution over outcomes
- Example: How long to get to the airport?

|              |        |   |        |   |        |                                                                                     |        |
|--------------|--------|---|--------|---|--------|-------------------------------------------------------------------------------------|--------|
| Time:        | 20 min |   | 30 min |   | 60 min |                                                                                     |        |
|              | x      |   | x      |   | x      |                                                                                     |        |
| Probability: | 0.25   | + | 0.50   | + | 0.25   |  | 35 min |

# What Probabilities to Use?

- In expectimax search, we have a probabilistic model of how the opponent (or environment) will behave in any state
  - Model could be a simple uniform distribution (roll a die)
  - Model could be sophisticated and require a great deal of computation
  - We have a chance node for any outcome out of our control: opponent or environment
  - The model might say that adversarial actions are likely!
- For now, assume each chance node magically comes along with probabilities that specify the distribution over its outcomes

# Modeling Assumption: Optimism vs Pessimism

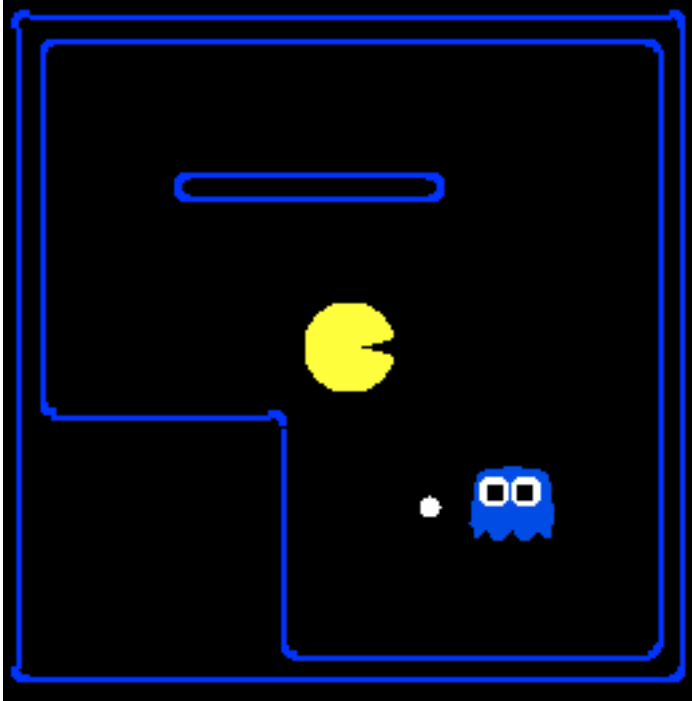
## Dangerous Optimism

- Assuming randomness when the world is adversarial

## Dangerous Pessimism

- Assuming the worst case when it's not likely

# Why should we average utilities?



|                   | Adversarial Ghost | Random Ghost |
|-------------------|-------------------|--------------|
| Minimax Pacman    |                   |              |
| Expectimax Pacman |                   |              |

- Pacman used depth 4 search with an eval function that avoids trouble
- Ghost used depth 2 search with an eval function that seeks Pacman